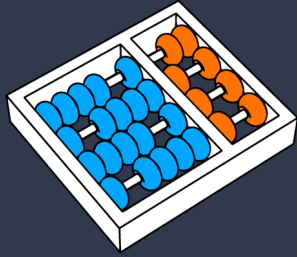




UNICAMP



Avaliação de Sistemas Multiagentes

Prof. Dr. Julio Cesar dos Reis

dosreis@unicamp.br

Agenda

- ❑ Avaliação de MAS
- ❑ Benchmarks
- ❑ Frameworks de avaliação
- ❑ Dimensões da Avaliação
- ❑ Taxonomia de Falhas
- ❑ Boas Práticas e Desafios

Motivação



Usuário:

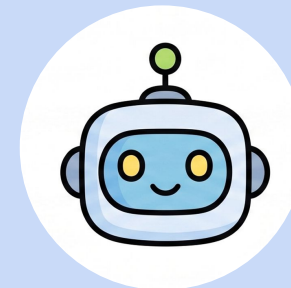
Reserve um voo de Madrid a Lima em julho, sem passar de 1000 EUR.

Motivação



Usuário:

Reserve um voo de Madrid a Lima em julho, sem passar de 1000 EUR.



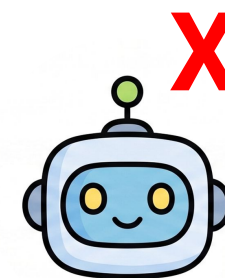
Agente:

Olá, reservei um voo por 900 EUR.

Problema

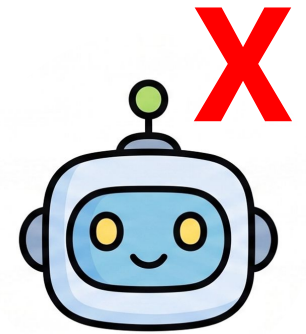
O agente informou ao usuário que a reserva do voo **havia sido concluída**.

No entanto, ao verificar o **banco de dados**, constatamos que a confirmação da reserva não foi finalizada, pois **não havia nenhum voo** disponível com custo inferior a 1000 EUR.



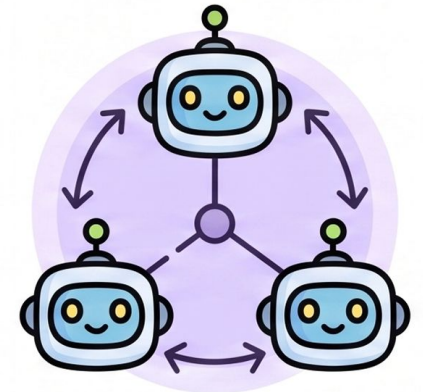
Por que a falha ocorreu?

- ❑ Um único agente tem **muita** responsabilidade.
 - Ele precisa verificar se a rota existe.
 - Verificar a disponibilidade para a data solicitada pelo usuário.
 - Verificar se o custo atende à restrição de orçamento definida pelo usuário.
- ❑ Devido à quantidade de ferramentas e às restrições de cada subtarefa, torna-se **difícil** depurar, avaliar e aprimorar a solução.
- ❑ **Separar papéis** muda o que se pode **medir**.



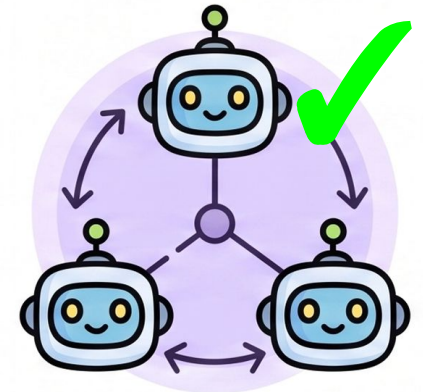
Como podemos melhorar essa tarefa?

- ❑ Com um Sistema Multiagente (MAS), as vantagens são:
 - As responsabilidades podem ser divididas entre subagentes.
 - Cada subagente pode ser avaliado individualmente.
 - Um subagente grava a reserva no banco de dados.
 - Outro gerencia a disponibilidade de assentos e a movimentação financeira.
 - Outro pode lidar com alterações, como duplicar ou cancelar uma reserva.

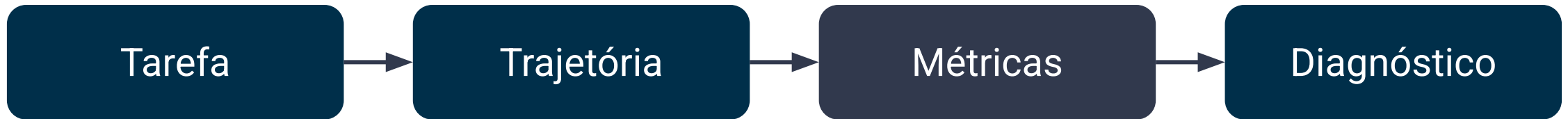


Avaliação de MAS

- ❑ Um MAS precisa ser avaliado com base no “estado” (contexto) do ambiente em que interage



Fluxo de avaliação MAS



Avaliação de MAS

- ❑ Mede capacidade, custo e confiabilidade de um sistema de agentes.
- ❑ Inspeciona a trajetória inteira, não apenas a resposta final
- ❑ Atribui responsabilidade a agentes e tool calls

Avaliação

- ❏ Avaliação roda *offline* sobre um conjunto fixo de tarefas
 - Diferentes versões são comparadas em condições controladas
 - Os **benchmarks** são criados a partir de:
 - um conjunto fixo de tarefas
 - cenários representativos do problema

Monitoramento

- ❑ Observa o sistema em produção, sem *ground truth*
 - As trajetórias coletadas podem servir de base para construir um dataset de avaliação

Benchmarks de Avaliação

Benchmark

- ❑ Fixa tarefas, ambiente, *ground truth* e métricas
 - Compara sistemas em um ranking
- ❑ Existem diferentes benchmarks, cada um voltado para avaliar capacidades específicas dos modelos

Panorama de Benchmarks

Benchmark	O que testa	Verificação
GAIA	Assistente generalista com browsing e tools	Resposta exata
SWE-bench	Resolver issues reais de GitHub	Testes unitários
τ-bench	Reserva sob regras de negócio	Hash do banco
MultiAgentBench	Colaboração e competição entre agentes	Milestones + LLM-judge
MCP-Atlas	Tool use em servidores MCP reais	Rubrica de claims

Panorama de Benchmarks

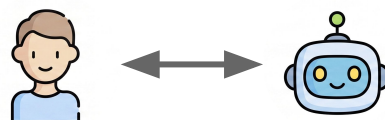
A métrica de saída desses benchmarks é a utilizada para reportar o desempenho de um novo modelo quando ele é disponibilizado.

Agentic		
Benchmark	Score	Versus best verified row
Terminal-Bench 2.0	88.3%	 Best verified: GPT-5.6 Sol • 91.9%
BrowseComp	91.2%	 Best verified: GPT-5.6 Sol • 92.2%
DeepSearchQA	95.0%	 Best verified: Claude Opus 5 • 95.0%
Toolathlon-Verified	73.2%	 Best verified: Claude Opus 5 • 80.6%
MCP Atlas	84.2%	 Best verified: Muse Spark 1.1 • 88.1%

<https://benchlm.ai/models/kimi-3>

Benchmark τ -bench

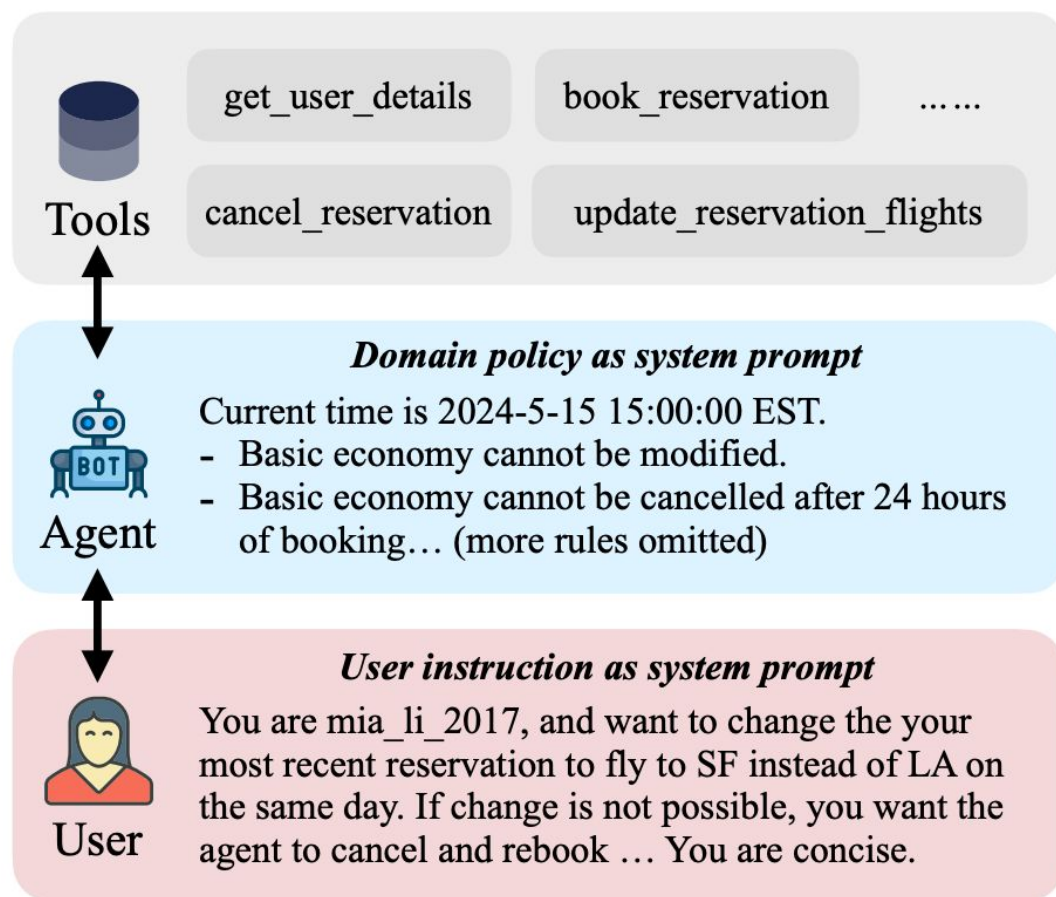
- ❑ Conversas dinâmicas entre usuário $\leftrightarrow$ agente $\leftrightarrow$ ferramenta em dois domínios (varejo e aviação)
- ❑ O agente deve satisfazer as consultas do usuário
- ❑ Cumprir as políticas definidas (regras de reembolso e prazos de cancelamento)



O agente é capaz de seguir as regras?

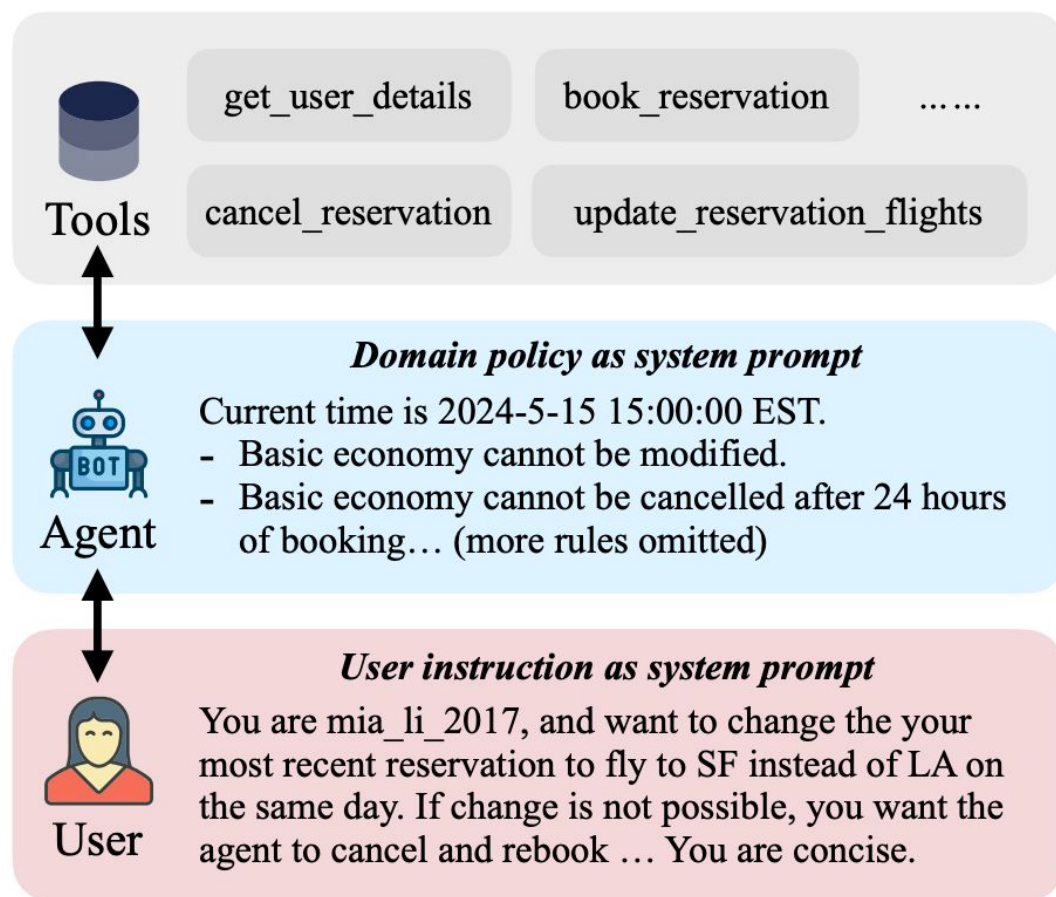
Benchmark τ -bench

(a) τ -bench setup

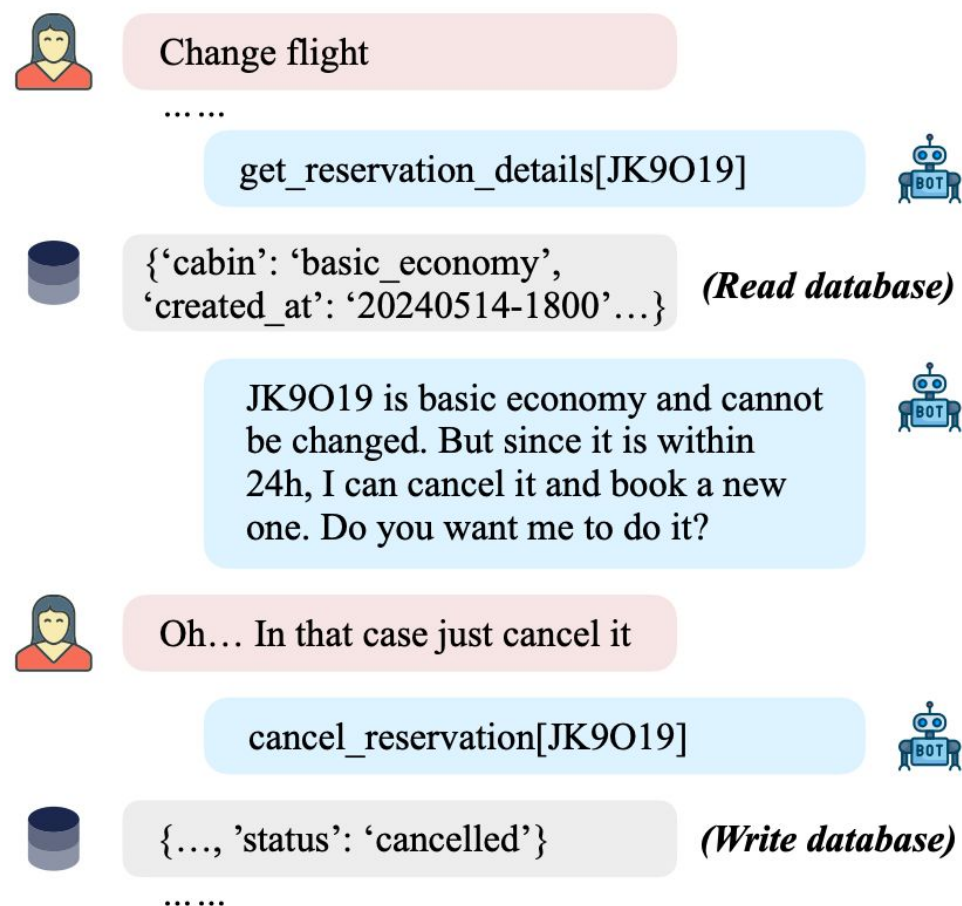


Benchmark τ -bench

(a) τ -bench setup



(b) Example trajectory in τ -airline



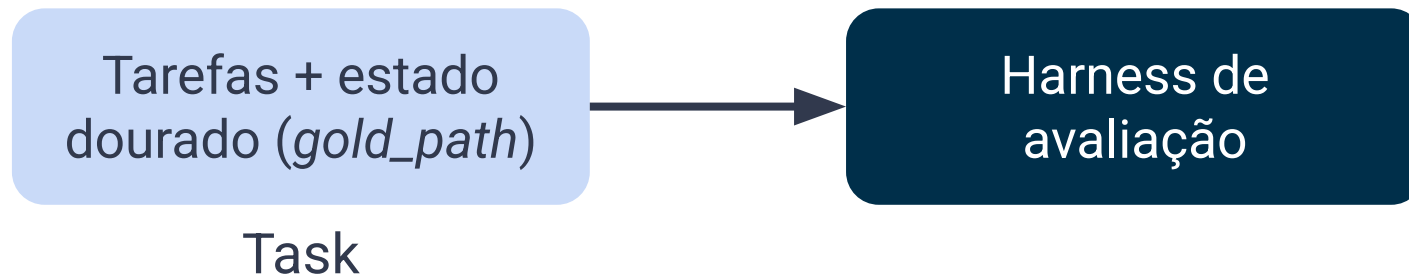
Frameworks de avaliação

Framework

- ❑ Software que roda sobre o seu sistema de agentes.
 - Instrumenta, avalia, versiona e executa testes
- ❑ Benchmark é a prova
- ❑ Framework é a infraestrutura de correção

Framework de avaliação

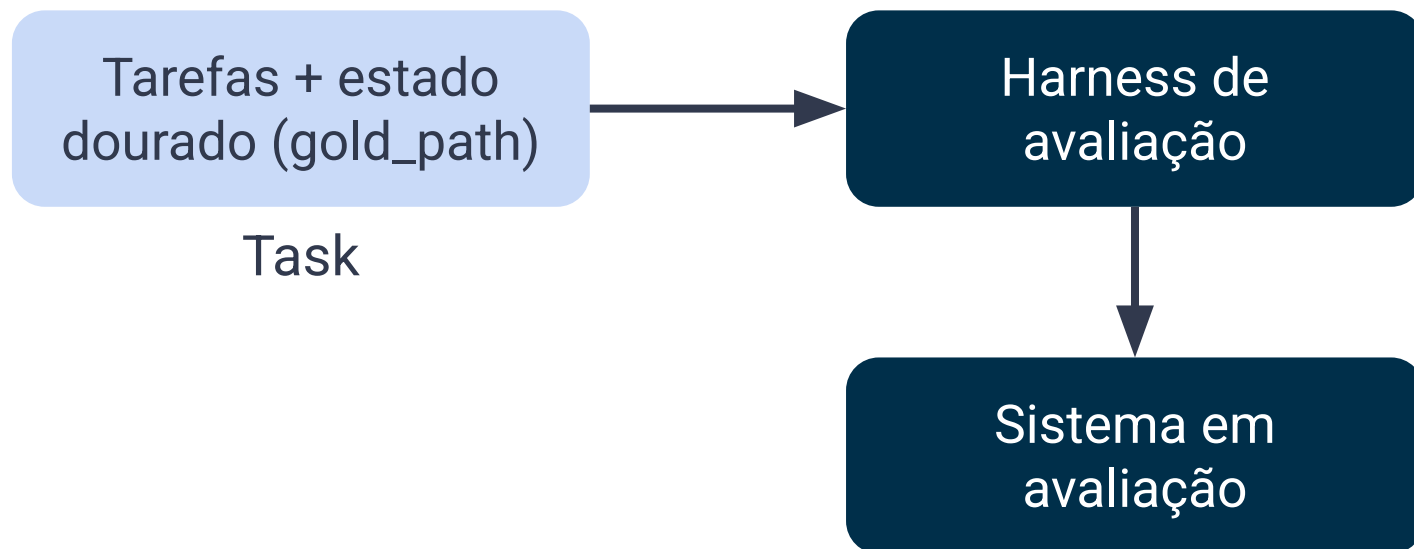
- ❑ O mesmo harness (framework) roda o benchmark - tarefa por tarefa



O framework carrega as tarefas do benchmark

Framework de avaliação

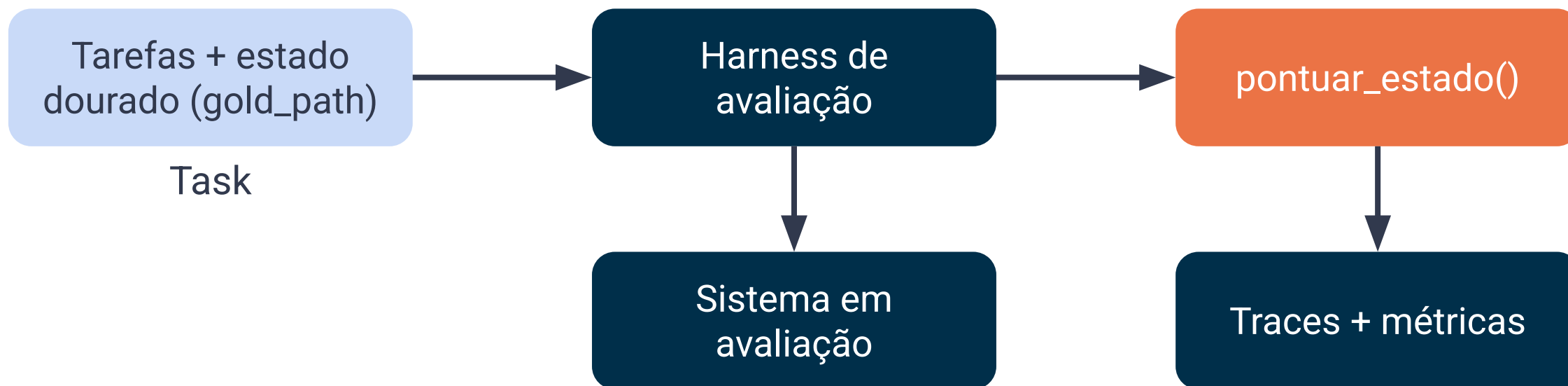
- ❑ O mesmo harness roda o benchmark, tarefa por tarefa.



Executa o sistema em sandbox e captura a trajetória

Framework de avaliação

- ❑ O mesmo harness (framework) roda o benchmark - tarefa por tarefa



Pontua o resultado e agrega em traces auditáveis

Panorama

Categoria	O que faz	Frameworks
Orquestração	Constrói o sistema: papéis, handoffs e loop de tools	LangGraph, CrewAI, AutoGen.
Eval harness	Carrega tarefas roda em sandbox pontua e agrega	Inspect AI, DeepEval, promptfoo.
Tracing	Captura cada chamada, argumento, token e latência	Langfuse, Phoenix, LangSmith, Opik.

Eval Harness

- ❑ Não existe uma “melhor” ferramenta.
- ❑ Cada uma é mais adequada dependendo do caso de uso e da utilidade que se busca.
- ❑ Para os eval harnesses, cada benchmark pode ter seu próprio harness ou ter sido desenvolvido sem o uso de uma ferramenta específica.

Tracing

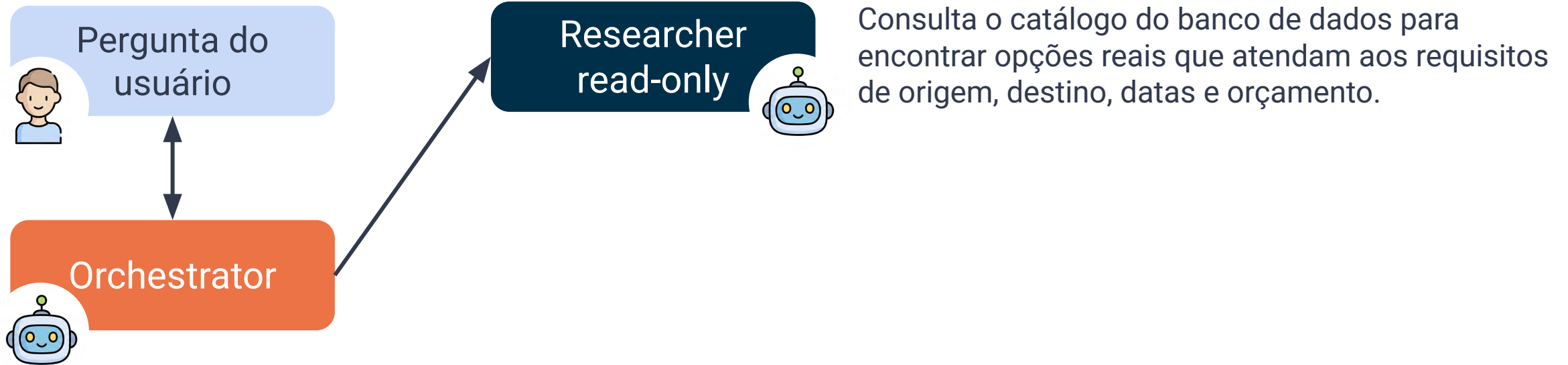
- ❑ Muitas ferramentas oferecem funcionalidades semelhantes.
- ❑ As principais diferenças estão em a solução ser open source, possuir licença para uso comercial e permitir self-hosting, eliminando a dependência de serviços em nuvem.

Dimensões da Avaliação

Exemplo

SubAgent

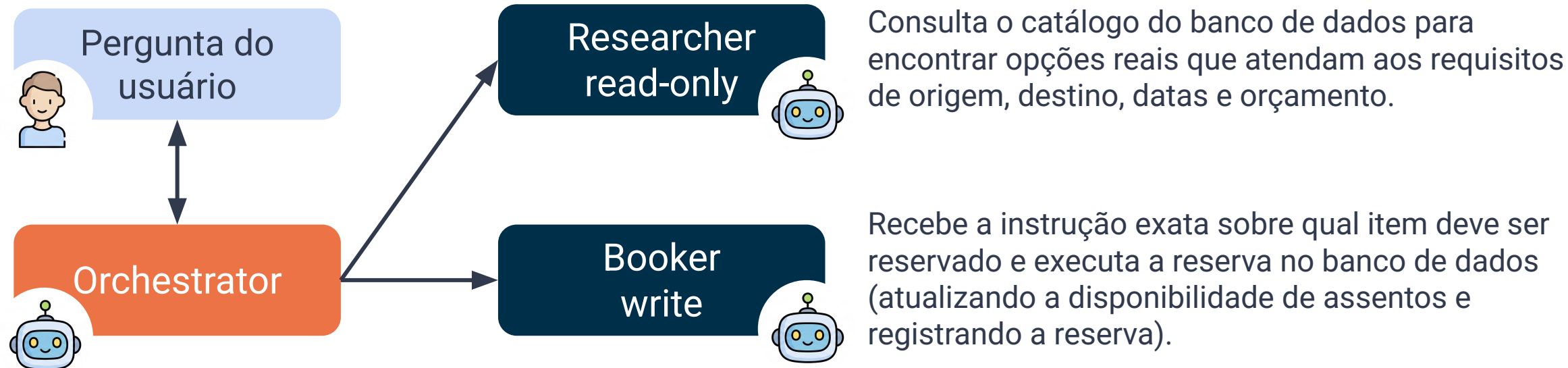
Tools



Exemplo

SubAgent

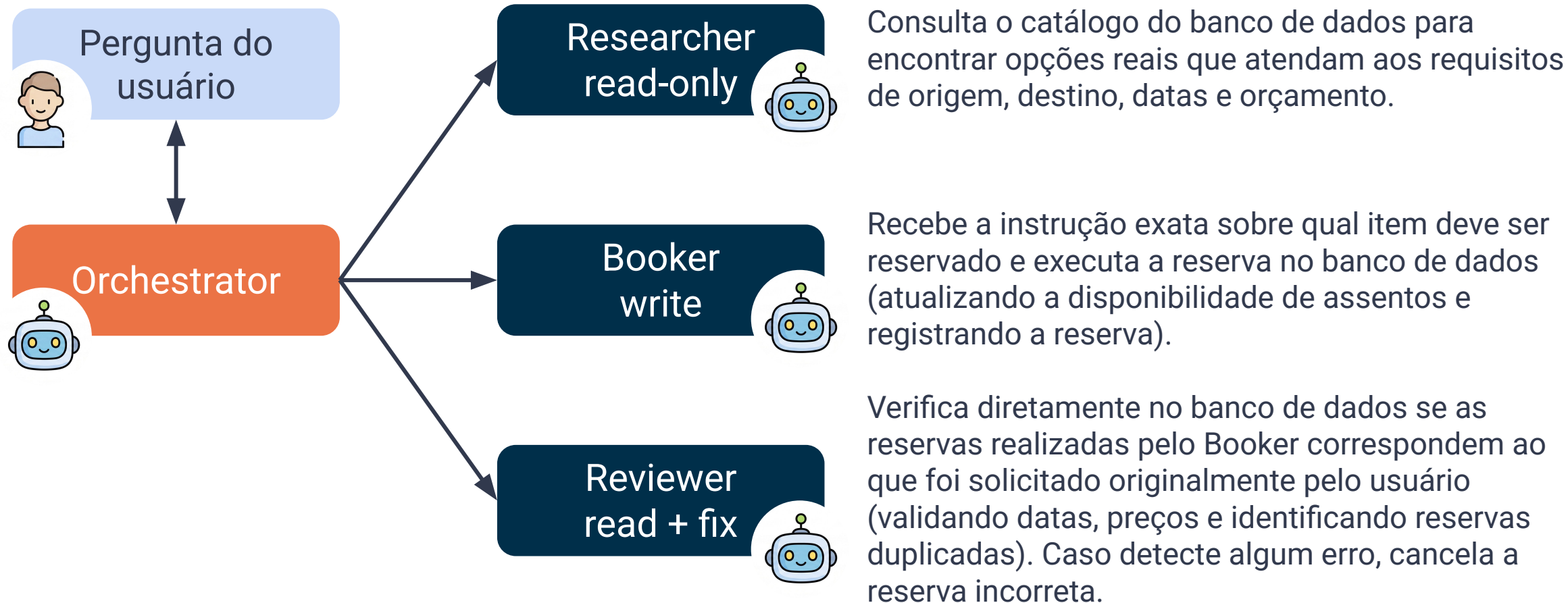
Tools



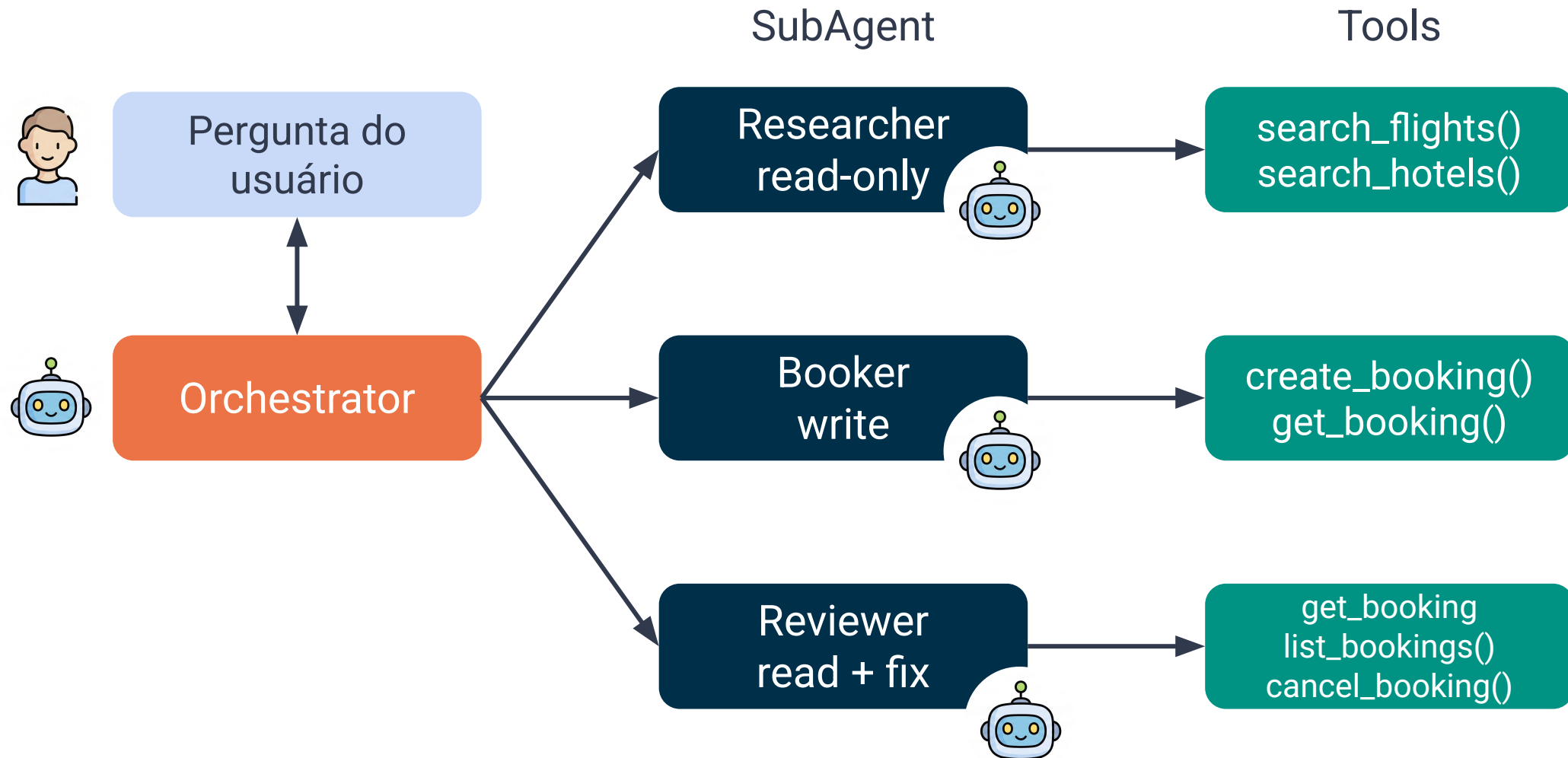
Exemplo

SubAgent

Tools



Exemplo



Taxonomia das Dimensões de Avaliação

❏ **Qualidade / sucesso na tarefa**

Taxa de sucesso / conclusão da tarefa; verificadores programáticos ou de correspondência exata; alcance de milestones e KPIs [1].

❏ **Eficiência**

Uso de tokens, custo monetário, número de passos/turnos, chamadas de tools.

[1] <https://aclanthology.org/2025.acl-long.421/>

[2] <https://arxiv.org/abs/2406.12045>

Taxonomia das Dimensões de Avaliação

❏ **Robustez / Confiabilidade**

Rode a tarefa k vezes.

- $\text{pass}@k$ basta 1 acerto: mede capacidade, sobe com k
- pass^k todos os k acertam: mede confiabilidade, cai com k

❏ **Colaboração / coordenação (específico de MAS)**

Eficiência da comunicação, divisão de trabalho, qualidade da coordenação, contribuição individual vs coletiva.

[1] <https://aclanthology.org/2025.acl-long.421/>

[2] <https://arxiv.org/abs/2406.12045>

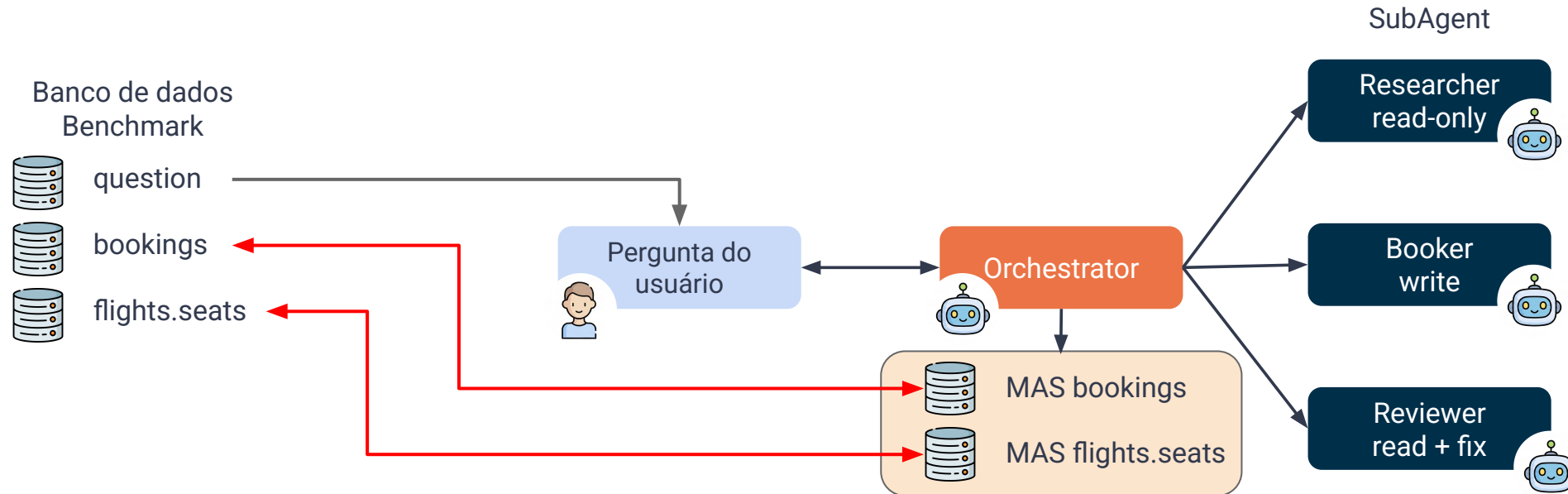
Qualidade (*Task Success*)

- ❑ Mede conclusão da tarefa, não fluência da resposta que o agente pode ter
 - Verificação programática do estado final do ambiente
 - Milestones intermediários quando a tarefa é longa

Qualidade (*Task Success*)

- ❑ Compara os resultados dos bancos de dados bookings e flights.seats com os gerados pelo MAS.
- ❑ A métrica utilizada é **PASS/FAIL**
- ❑ Indica se os valores gerados correspondem exatamente aos armazenados no banco de dados

Qualidade (Task Success)



Eficiência

- ❑ Contabiliza tokens, custo monetário, passos e latência
 - Penaliza tool calls redundantes e mensagens sem efeito
- ❑ Compara arquiteturas sob o mesmo orçamento
- ❑ Acurácia sem custo declarado não é resultado

Eficiência (*trajectory*)

Consideremos este caso:

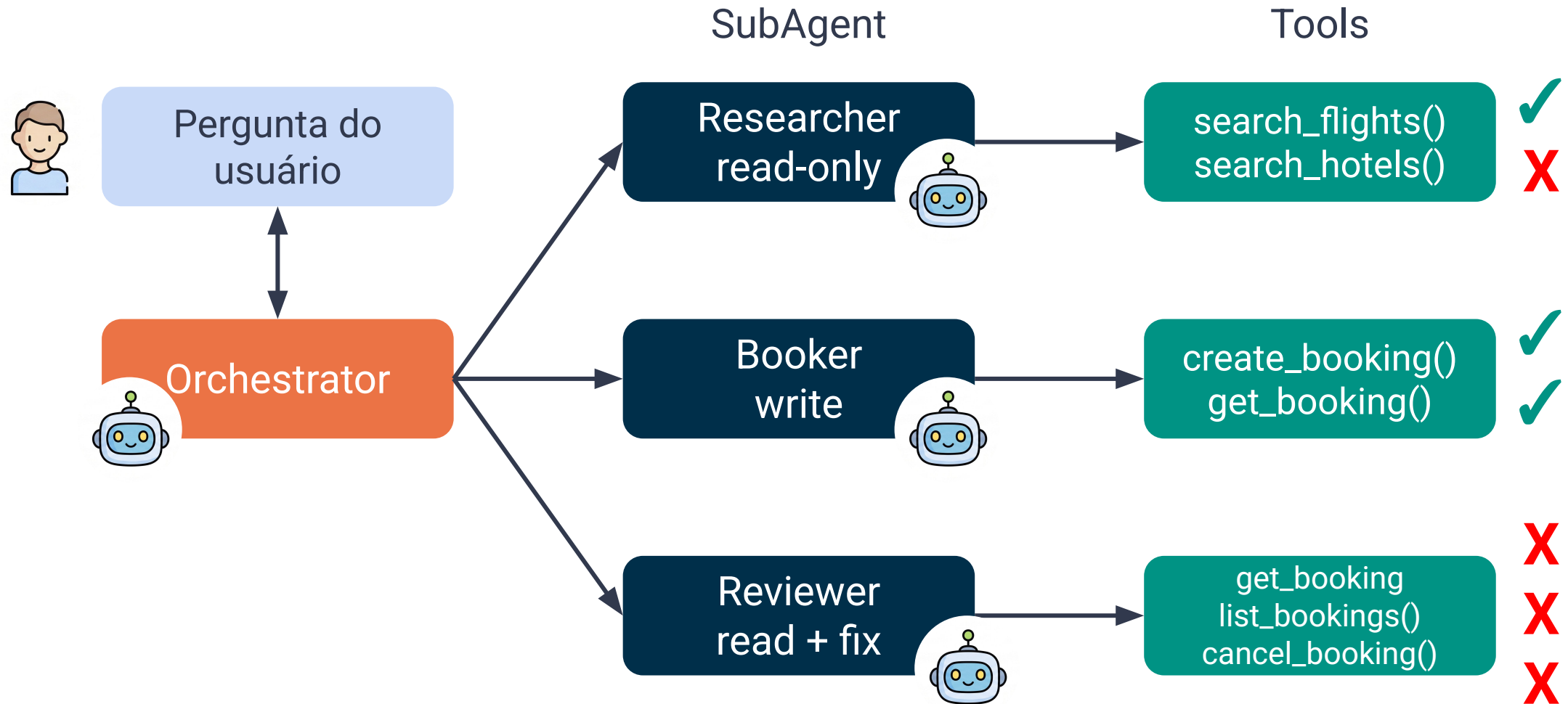


Sou o usuário u-42. Quero voar de Madri (MAD) para Lima (LIM) no dia 12/09/2026, com um orçamento máximo de EUR 1.000. Faça a reserva.

Para resolvê-lo, temos o *gold path* de ferramentas (tools) que o MAS deve seguir:

```
"reference": [  
  ("search_flights", {"origin": "MAD", "destination": "LIM", "date": "2026-09-12",  
    "max_price": 1000.0}),  
  ("create_booking", {"user_id": "u-42", "kind": "flight", "item_id": "FL-101",  
    "start_date": "2026-09-12", "price": 980.0}),  
  ("get_booking", {"booking_id": "BK-0001"}),  
],
```

Eficiência (*trajectory*)



Eficiência (*trajectory*)

- ❑ O gold path representa a sequência mínima e indispensável de ações que o MAS deve executar para concluir a tarefa com sucesso.

1. *Tool search_flights (do Researcher)*
2. *Tool create_booking (do Booker)*
3. *Tool get_booking (do Booker)*

Eficiência (*trajectory*)

Medimos as trajetórias geradas pelo MAS sob diferentes aspectos

Modo	Passa se	Quando usar
<i>strict</i>	mesma sequência, mesma ordem, sem chamadas extras	fluxos rígidos e auditáveis
<i>unordered</i>	mesmas chamadas, ordem livre	quando a ordem não é semanticamente relevante
<i>subset</i>	o agente executou no máximo o que está na referência	detectar etapas extras
<i>superset</i>	o agente executou pelo menos o que está na referência	tolerar verificações adicionais

Eficiência (*trajectory*)

Case	Description	Trajectory	strict	unordered	subset	superset
Execução idêntica	O agente executa exatamente os três passos, na mesma ordem e utilizando os mesmos argumentos nas chamadas de escrita.	<i>[search_flights → create_booking → get_booking]</i>	✓	✓	✓	✓

Modo	Passa se	Quando usar
strict	mesma sequência, mesma ordem, sem chamadas extras	fluxos rígidos e auditáveis
unordered	mesmas chamadas, ordem livre	quando a ordem não é semanticamente relevante
subset	o agente executou no máximo o que está na referência	detectar etapas extras
superset	o agente executou pelo menos o que está na referência	tolerar verificações adicionais

Reference: *[search_flights → create_booking → get_booking]*

Eficiência (*trajectory*)

Case	Description	Trajectory	strict	unordered	subset	superset
O agente executa uma verificação adicional	O agente executa os três passos, mas o subagente Reviewer faz uma chamada adicional para list_bookings para garantir que o usuário não tenha reservas duplicadas.	[search_flights → create_booking → get_booking → list_bookings]	X	X	X	✓

Modo	Passa se	Quando usar
strict	mesma sequência, mesma ordem, sem chamadas extras	fluxos rígidos e auditáveis
unordered	mesmas chamadas, ordem livre	quando a ordem não é semanticamente relevante
subset	o agente executou no máximo o que está na referência	detectar etapas extras
superset	o agente executou pelo menos o que está na referência	tolerar verificações adicionais

Reference: [search_flights → create_booking → get_booking]

Eficiência (*trajectory*)

Case	Description	Trajectory	strict	unordered	subset	superset
O agente executa as chamadas em uma ordem diferente.	O agente executa as mesmas três ferramentas, mas em uma ordem diferente (por exemplo, buscou o voo no final ou em paralelo).	<i>[create_booking → search_flights → get_booking]</i>	X	✓	✓	✓

Modo	Passa se	Quando usar
strict	mesma sequência, mesma ordem, sem chamadas extras	fluxos rígidos e auditáveis
unordered	mesmas chamadas, ordem livre	quando a ordem não é semanticamente relevante
subset	o agente executou no máximo o que está na referência	detectar etapas extras
superset	o agente executou pelo menos o que está na referência	tolerar verificações adicionais

Reference: [search_flights → create_booking → get_booking]

Eficiência (*trajectory*)

Case	Description	Trajectory	strict	unordered	subset	superset
O agente omite uma etapa obrigatória.	O agente encontra o voo e cria a reserva, mas se esquece de executar get_booking para confirmar a leitura.	[search_flights → create_booking]	X	X	✓	X

Modo	Passa se	Quando usar
strict	mesma sequência, mesma ordem, sem chamadas extras	fluxos rígidos e auditáveis
unordered	mesmas chamadas, ordem livre	quando a ordem não é semanticamente relevante
subset	o agente executou no máximo o que está na referência	detectar etapas extras
superset	o agente executou pelo menos o que está na referência	tolerar verificações adicionais

Reference: [search_flights → create_booking → get_booking]

Eficiência (*trajectory*)

- ❑ Avaliar trajetórias em MAS requer ajustar a métrica de acordo com o nível de rigidez ou autonomia permitido
- ❑ Não existe uma única métrica “correta”
- ❑ Há uma métrica adequada para cada intenção/necessidade de avaliação.

Robustez (*Reliability*)

- ❑ Repete a mesma tarefa k vezes e reporta variância
 - $\text{pass}@k$ mede capacidade
 - pass^k mede confiabilidade
- ❑ Perturba entradas, tools que falham e injeções adversárias

$\text{pass}@k$ basta 1 acerto: mede capacidade, sobe com k

pass^k todos os k acertam: mede confiabilidade, cai com k

Colaboração (MAS-specific)

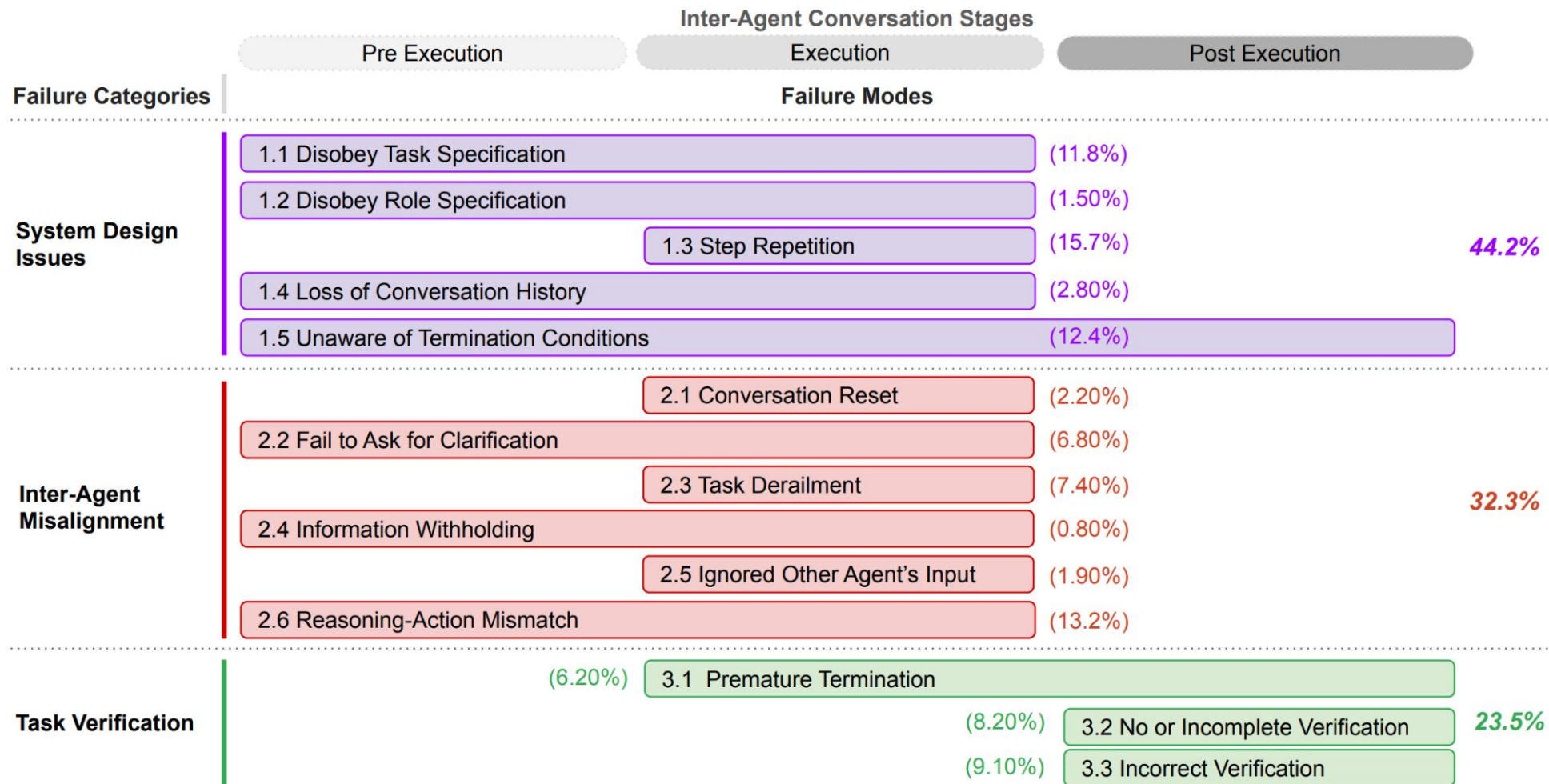
- ❑ Mede tokens gastos em delegação, não em tarefa
- ❑ Detecta papéis sobrepostos: Researcher e Reviewer lendo o mesmo dado.
- ❑ Verifica se o fluxo obrigatório foi respeitado
- ❑ Rastreia qual subagente causou a falha

Taxonomia de Falhas

Taxonomia de Falhas

- ❑ Derivada de trajetórias reais da execução de frameworks populares
- ❑ Falhas mais comuns:
 - **Especificação:** papéis mal definidos, instruções violadas
 - **Desalinhamento:** agentes se ignoram ou se contradizem
 - **Verificação:** o sistema encerra sem checar o resultado
- ❑ Boa parte das falhas é organizacional, não do modelo

Taxonomia de Falhas



Boas Práticas e Desafios

Boas Práticas na Avaliação

- ❑ Varia um fator por vez: modelo fixo ao comparar arquiteturas
- ❑ Reporta custo junto da acurácia
- ❑ Roda k trials isolados
- ❑ Publica $\text{pass}@k$ e pass^k com n e k
- ❑ Registra versões, prompts, configs e trajetórias liberadas

Desafios

❏ Contaminação

- Tarefas públicas vazam para o treino e “inflam” o placar

❏ *Reward hacking*

- O sistema otimiza a brecha do grader, não a tarefa

Síntese

- ❑ Avaliação é um dos pilares centrais dos MAS baseados em LLMs
- ❑ Transforma trajetórias em estado verificável e diagnóstico
- ❑ Diferentes *graders* cada um adequado a diferentes cenários
- ❑ Em ambientes reais, pass^k e custo precisam ser reportados